

Guía Técnica de Referencia

RAG · PostgreSQL vs MySQL · Pandas · AI Training · Terraform · CD con Azure
On-Premise

2025 — Edición Profesional

Índice de contenidos

1. RAG y sus variantes
2. Cuándo aplicar RAG y cuándo no
3. PostgreSQL vs MySQL
4. Pandas (Python)
5. Recomendaciones para entrenar IA
6. Terraform — qué es, cómo y cuándo usarlo
7. CD con Azure + On-Premise

1. RAG y sus variantes

RAG (Retrieval-Augmented Generation) es una arquitectura que combina un modelo de lenguaje (LLM) con un sistema de recuperación de información externo. En lugar de depender únicamente del conocimiento interno del modelo, RAG recupera fragmentos relevantes de una base de conocimiento y los inyecta como contexto antes de generar la respuesta.

Flujo básico de RAG

Query → Embedding → Vector Search → Context chunks → LLM → Respuesta

Variantes principales

Variante	Descripción	Caso de uso típico
Naive RAG	Recuperación simple por similitud coseno sobre un vector store.	FAQs, documentación estática
Advanced RAG	Añade re-ranking, query expansion y chunking inteligente.	Contratos, manuales técnicos
Modular RAG	Pipeline configurable: múltiples retrievers, filtros y memorias.	Asistentes empresariales
Graph RAG	Usa grafos de conocimiento para recuperar relaciones entre entidades.	Investigación, KGs complejos
Self-RAG	El LLM decide cuándo y qué recuperar de forma autónoma.	Agentes autónomos
Corrective RAG (CRAG)	Evalúa la relevancia de los chunks y corrige si son insuficientes.	Alta precisión requerida
Agentic RAG	El agente orquesta múltiples herramientas de recuperación.	Flujos multi-fuente
HyDE	Genera un documento hipotético y lo usa como query de búsqueda.	Preguntas abstractas

2. Cuándo aplicar RAG y cuándo NO

RAG no es la solución universal. La decisión correcta depende del tipo de conocimiento, la frecuencia de actualización y los requisitos de latencia.

■ Cuándo Sí usar	■ Cuándo NO usar
• Conocimiento que cambia frecuentemente	• El conocimiento cabe en el contexto del LLM
• Documentación privada o confidencial	• Latencia crítica (< 200 ms)
• Respuestas que deben citar fuentes	• Preguntas de razonamiento puro sin hechos
• Base de conocimiento > 100 k tokens	• Datos muy desestructurados sin semántica clara
• Dominio muy específico (legal, médico, interno)	• Prototipo rápido sin infraestructura vector
• Necesitas trazabilidad (qué fuente usó el modelo)	• El modelo ya fue fine-tuned con esos datos

■ Regla práctica: Si el dato cambia más rápido que tu ciclo de fine-tuning, usa RAG.

3. PostgreSQL vs MySQL

Ambos son RDBMS open-source maduros. La elección correcta depende del tipo de workload, la complejidad de las consultas y el ecosistema.

Característica	PostgreSQL	MySQL
Licencia	PostgreSQL License (permisiva)	GPL / MySQL Commercial
Conformidad SQL	Muy alta (casi completa)	Parcial (modo estricto opcional)
Tipos de datos	JSON, JSONB, arrays, hstore, geoespacial	JSON básico (sin índices nativos en < 8.0)
Rendimiento lecturas simples	Muy bueno	Excelente (InnoDB optimizado)
Consultas complejas / analíticas	Excelente (CTE recursivas, window fns)	Limitado
Full-text search	Nativo avanzado	Básico
Extensiones	PostGIS, pgvector, TimescaleDB, etc.	Limitadas
Replicación	Lógica y física nativa	Binlog, Group Replication
Concurrencia (MVCC)	MVCC nativo sin bloqueos de lectura	MVCC con bloqueos en ciertos casos
Comunidad/Ecosistema	Amplia, especialmente en cloud	Muy amplia, dominante en LAMP
Ideal para	Apps complejas, ML, geoespacial, fintech	CMS, e-commerce, LAMP stack

■ pgvector convierte a PostgreSQL en un vector store. Ideal para RAG sin infraestructura adicional.

4. Pandas (Python)

Pandas es la librería de facto para manipulación y análisis de datos tabulares en Python. Provee las estructuras DataFrame y Series sobre NumPy, con una API expresiva para ETL, exploración y preparación de datos.

Operaciones esenciales

Cargar datos

```
df = pd.read_csv('data.csv') # también read_excel, read_json, read_sql
```

Inspección rápida

```
df.head() | df.info() | df.describe() | df.shape
```

Filtrado

```
df[df['edad'] > 30] → df.query('edad > 30')
```

Selección columnas

```
df[['col1', 'col2']] → df.loc[:, 'col1':'col3']
```

Agrupación

```
df.groupby('categoria')['ventas'].sum()
```

Merge / Join

```
pd.merge(df1, df2, on='id', how='left')
```

Valores nulos

```
df.fillna(0) | df.dropna(subset=['col'])
```

Transformación

```
df['nueva'] = df['col'].apply(lambda x: x*2)
```

Pivot table

```
df.pivot_table(values='v', index='a', columns='b', aggfunc='sum')
```

Exportar

```
df.to_csv('out.csv', index=False) | df.to_parquet('out.parquet')
```

Cuándo usar Pandas vs alternativas

Herramienta	Cuándo usarla
Pandas	Datos < 1–2 GB en memoria, exploración, ETL local
Polars	Datos medianos-grandes, rendimiento crítico, multi-hilo
Dask	Datos que no caben en RAM, paralelización en cluster
PySpark	Big Data distribuido (GB a PB), pipelines productivos

DuckDB

Consultas SQL analíticas sobre archivos
CSV/Parquet locales

5. Recomendaciones para entrenar IA

El entrenamiento de modelos de IA va mucho más allá de elegir un algoritmo. Estas son las mejores prácticas consolidadas para proyectos de ML/DL en producción.

Datos

- Calidad > cantidad: limpia, balancea y valida antes de entrenar.
- Versiona tus datasets (DVC, Delta Lake) junto al código.
- Aplica data augmentation para reducir overfitting.
- Separa siempre train / validation / test (80-10-10 o estratificado).
- Detecta y elimina data leakage antes de cualquier experimento.

Modelo y arquitectura

- Empieza con un baseline simple (regresión, árbol) antes de deep learning.
- Fine-tuning > entrenar desde cero cuando hay modelos pre-entrenados disponibles.
- Usa transfer learning: reduce tiempo y datos necesarios.
- Regulariza: dropout, L1/L2, batch normalization, early stopping.
- Evalúa con métricas del negocio, no solo accuracy.

Infraestructura y MLOps

- Trackea todos los experimentos: MLflow, Weights & Biases, Comet.
- Usa contenedores Docker para reproducibilidad del entorno.
- Automatiza el pipeline: data → train → eval → deploy (CI/CD for ML).
- Monitorea el modelo en producción: data drift, concept drift.
- Implementa model registry para gestionar versiones de modelos.

LLMs / Fine-tuning

- Prefiere RAG o prompting antes de fine-tuning si el problema lo permite.
- Para fine-tuning usa LoRA / QLoRA para reducir recursos.
- Mínimo ~500–1000 ejemplos de alta calidad para fine-tuning efectivo.
- Evalúa con benchmarks específicos del dominio, no solo MMLU.
- Guarda checkpoints frecuentes; los entrenamientos largos pueden fallar.

6. Terraform — qué es, cómo y cuándo usarlo

Terraform es una herramienta de Infrastructure as Code (IaC) de HashiCorp que permite definir, provisionar y gestionar infraestructura en múltiples proveedores cloud (Azure, AWS, GCP, on-premise) mediante archivos declarativos HCL.

Conceptos clave

Concepto	Descripción
Provider	Plugin que conecta Terraform con un proveedor (azurerm, aws, google...)
Resource	Unidad de infraestructura (VM, BD, red, grupo de recursos...)
State	Archivo .tfstate que mapea el mundo real con la config declarada
Module	Conjunto reutilizable de recursos con inputs/outputs
Workspace	Entornos aislados (dev, staging, prod) sobre el mismo código
Plan / Apply	Plan muestra cambios; Apply los ejecuta de forma segura

Flujo de trabajo

```
terraform init # descarga providers y módulos terraform plan # previsualiza cambios (dry-run) terraform apply # aplica los cambios terraform destroy # destruye la infraestructura
```

Ejemplo mínimo — Resource Group en Azure

```
provider "azurerm" { features {} } resource "azurerm_resource_group" "main" { name = "mi-app-rg" location = "West Europe" }
```

Cuándo usar Terraform

■ Cuándo SÍ usar	■ Cuándo NO usar
• Infraestructura multi-cloud o híbrida	• Un solo recurso puntual (usa CLI o portal)
• Equipos que necesitan reproducibilidad	• Equipo sin conocimiento de IaC (curva inicial)
• Entornos efímeros (dev/test/staging)	• Config muy dinámica (considera Pulumi o CDK)
• Auditoría y revisión de cambios vía PR	• App PaaS sin infraestructura custom

- Migración o replicación de entornos

■ Guarda el tfstate en un backend remoto (Azure Blob, S3) y activa el bloqueo de estado para trabajo en equipo.

7. CD con Azure + On-Premise

Implementar Continuous Delivery (CD) en entornos híbridos (Azure + servidores on-premise) requiere decisiones de conectividad, herramientas de despliegue y estrategias de release acordes con los SLAs y la topología de red.

Cuándo aplicar CD híbrido

■ Cuándo Sí usar	■ Cuándo NO usar
• Apps con componentes en cloud y on-prem simultáneamente	• App 100% en Azure: usa solo Azure DevOps/GitHub Actions
• Regulación obliga a mantener datos on-premise	• App 100% on-prem sin plan de cloud: usa Jenkins/GitLab CI
• Latencia crítica requiere ejecución local	• Infraestructura sin conectividad estable a internet
• Migración gradual a cloud (lift & shift progresivo)	
• Edge computing con sync a Azure	

Arquitectura de referencia

```
GitHub / Azure Repos ↓ Azure DevOps Pipeline (CI: build, test, push image) ↓ Azure Container Registry ← imagen versionada ↓ ↓ Azure (AKS / App Service) Self-hosted Agent (on-prem) ↓ Deploy vía Ansible / Helm
```

Herramientas recomendadas por capa

Capa	Herramienta	Notas
Pipeline CI/CD	Azure DevOps / GitHub Actions	Soporta self-hosted runners nativamente
Agente on-prem	Azure DevOps Self-hosted Agent	Instalar en VM on-prem, acceso al entorno local
Artefactos	Azure Container Registry (ACR)	Centraliza imágenes accesibles desde ambos lados
Config/Secrets	Azure Key Vault + env vars	Referencia desde pipeline; inyectar en tiempo de deploy
IaC	Terraform + azurerm provider	Gestiona recursos cloud; Ansible para on-prem
Monitoreo	Azure Monitor + Prometheus	Arc-enabled servers para on-prem en el mismo dashboard

Conectividad	Azure Arc / VPN Gateway / ExpressRoute	Arc permite gestionar VMs on-prem como si fueran Azure
--------------	--	--

Estrategias de release recomendadas

- Blue/Green: despliega en entorno paralelo y cambia el tráfico. Mínimo downtime.
- Canary: libera a un % de usuarios primero. Ideal para validar en producción.
- Rolling update: actualiza instancias gradualmente. Nativo en Kubernetes.
- Feature flags: desacopla deploy de release. Activa funcionalidades sin redespigue.

■ Azure Arc permite registrar servidores on-premise en Azure Resource Manager, habilitando políticas, monitoreo y GitOps desde el mismo plano de control cloud.